

基于 MindSpore 的线性函数拟合实验



华为技术有限公司

版权所有 © 华为技术有限公司 2024 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司		
地址：	深圳市龙岗区坂田华为总部办公楼	邮编：518129
网址：	http://e.huawei.com	



目录

1 实验介绍	2
1.1 实验目的	2
1.2 实验清单	2
1.3 实验开发环境	2
2 线性回归模拟实验	3
2.1 实验介绍	3
2.1.1 简介	3
2.2 实验环境要求	3
2.3 实验步骤	3
2.3.1 导入 MindSpore 模块和辅助模块	3
2.3.2 数据生成与加载	4
2.3.3 模型构建	4
2.3.4 模型训练	5
2.3.5 模型预测	6
2.3.6 可视化	7
2.4 实验小结	7

1 实验介绍

本实验主要介绍使用 MindSpore 在模拟数据上进行线性回归实验，分析自变量和因变量之间的线性关系，即求得一个线性函数。

1.1 实验目的

- 了解线性回归的基本概念和问题模拟；
- 了解如何使用 MindSpore 进行线性回归实验。

1.2 实验清单

实验	简述	难度	软件环境	开发环境
线性回归模拟实验	基于随机生成的数据，进行线性回归实验，实现对数据的拟合	中级	MindSpore2.2	ModelArts

1.3 实验开发环境

- MindSpore-2.2

本实验在华为云 ModelArts 上进行，开发环境搭建可参考如下手册：ModelArts 云环境环境搭建指南。



华为云ModelArts
环境搭建手册.docx

2 线性回归模拟实验

2.1 实验介绍

2.1.1 简介

线性回归 (Linear Regression) 是机器学习最经典的算法之一，具有如下特点：

- 自变量服从正态分布；
- 因变量是连续性数值变量；
- 自变量和因变量呈线性关系。

本实验主要介绍使用 MindSpore 在模拟数据上进行线性回归实验，分析自变量和因变量之间的线性关系，即求得一个线性函数。

2.2 实验环境要求

- MindSpore 2.2 (MindSpore 版本会定期更新，本指导也会定期刷新，与版本配套) ；
- 本案例支持 win_x86 和 Linux 系统，CPU/GPU/Ascend 均可运行；
- 华为云 ModelArts：ModelArts 是华为云提供的面向开发者的一站式 AI 开发平台，集成了昇腾 AI 处理器资源池，用户可以在该平台下体验 MindSpore。

2.3 实验步骤

2.3.1 导入 MindSpore 模块和辅助模块

代码：

```
import os
from PIL import Image
import numpy as np
import mindspore as ms
from mindspore import nn, Tensor, ops
from mindspore.dataset import transforms, vision, GeneratorDataset
```

```
ms.set_context(mode=ms.PYNATIVE_MODE, device_target="Ascend")    #使用云平台昇腾算力
```

2.3.2 数据生成与加载

根据线性函数 $y = -5 * x + 0.1$ 生成模拟数据，并在其中加入少许扰动。

代码：

```
# Iterable object as input source
class Iterable:
    def __init__(self):
        self._data = np.arange(-5, 5, 0.3)[:32].reshape((32, 1, 1))
        self._label = -5 * self._data + 0.1 * np.random.normal(loc=0.0, scale=20.0, size=self._data.shape)

    def __getitem__(self, index):
        return self._data[index], self._label[index]

    def __len__(self):
        return len(self._data)

data = Iterable()
dataset = GeneratorDataset(source=data, column_names=["data", "label"])
```

2.3.3 模型构建

使用 MindSpore 提供的 `nn.Dense(1, 1)` 算子作为线性模型，其中 (1, 1) 表示线性模型的输入和输出皆是 1 维，即 w 是 1×1 的矩阵。算子会随机初始化权重 w 和偏置 b 。

$$y = w * x + b$$

采用均方差 (Mean Squared Error, MSE) 作为损失函数。采用随机梯度下降 (Stochastic Gradient Descent, SGD) 对模型进行优化。

定义模型训练函数首先需要实例化回归模型、定义损失函数和优化器。由于需要使用函数式自动微分，需要将神经网络和损失函数的调用封装为一个前向计算函数。完成后，使用 `value_and_grad` 接口获得微分函数，用于计算梯度。定义 `train_step` 函数，计算梯度并用于优化，返回损失。

代码：

```
net = nn.Dense(1, 1)
loss_fn = nn.MSELoss()
opt = nn.SGD(net.trainable_params(), learning_rate=0.01)
def train_loop(model, dataset, loss_fn, optimizer):
    # 定义前向传播函数
    def forward_fn(data, label):
        logits = model(data)
        loss = loss_fn(logits, label)
```

```
        return loss, logits

# 获得梯度函数
grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters, has_aux=True)

# 定义 one-step 训练函数
def train_step(data, label):
    (loss, _), grads = grad_fn(data, label)
    loss = ops.depend(loss, optimizer(grads))
    return loss

# 返回一个 epoch 中的 batch 数。
size = dataset.get_dataset_size()
model.set_train()
for batch, (data, label) in enumerate(dataset.create_tuple_iterator()):
    loss = train_step(data.astype(ms.float32), label.astype(ms.float32))
    if batch % 10 == 0:
        loss, current = loss.asnumpy(), batch
        print(f"loss: {loss:>7f}   [{current:>3d}/{size:>3d}"])
```

2.3.4 模型训练

代码：

```
epochs = 20
batch_size = 1
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(net, dataset, loss_fn, opt)
print("Done!")
```

输出：

```
Epoch 1
-----
loss: 340.915100   [ 0/ 32]
loss: 1.483280    [ 10/ 32]
loss: 0.709444    [ 20/ 32]
loss: 11.089573   [ 30/ 32]
Epoch 2
-----
.....
Epoch 19
-----
loss: 0.313206    [ 0/ 32]
loss: 0.011601    [ 10/ 32]
loss: 0.136323    [ 20/ 32]
loss: 5.667728    [ 30/ 32]
Epoch 20
```

```
-----
loss: 0.456090 [ 0/ 32]
loss: 5.791370 [ 10/ 32]
loss: 8.842282 [ 20/ 32]
loss: 1.286176 [ 30/ 32]
Done!
```

2.3.5 模型预测

首先可以查看参数的名称、维度、数据类型、数值以及是否求导（当 `requires_grad=True` 时，参数需要求导，进行反向传播）。

```
net.trainable_params()
```

输出：

```
[Parameter (name=weight, shape=(1, 1), dtype=Float32, requires_grad=True),
 Parameter (name=bias, shape=(1,), dtype=Float32, requires_grad=True)]
```

查看第一个可训练参数：

```
net.trainable_params()[0]
```

输出：

```
Parameter (name=weight, shape=(1, 1), dtype=Float32, requires_grad=True)
```

将第一个参数数组化：

```
net.trainable_params()[0].asnumpy()
```

输出：

```
array([[ -5.3179183]], dtype=float32)
```

训练一定的轮次后，得到的模型已经十分接近真实的线性函数了，使用训练好的模型进行预测。

代码：

```
wb = [x.asnumpy() for x in net.trainable_params()]
w, b = np.squeeze(wb[0]), np.squeeze(wb[1])
print('The true linear function is y = -5 * x + 0.1')
print('The trained linear model is y = {0} * x + {1}'.format(w, b))

for i in range(-10, 11, 5):
    print('x = {0}, predicted y = {1}'.format(i, net(ms.Tensor([i]), ms.float32)))
```

输出：

```
The true linear function is y = -5 * x + 0.1
The trained linear model is y = -4.842680931091309 * x + 0.03442131727933884
x = -10, predicted y = [[49.714813]]
x = -5, predicted y = [[24.974724]]
x = 0, predicted y = [[0.23463698]]
```



```
x = 5, predicted y = [[-24.505451]]
x = 10, predicted y = [[-49.245537]]
```

2.3.6 可视化

模拟的样本数据、真实的线性函数和训练得到的线性模型，如下图所示：

代码：

```
from matplotlib import pyplot as plt
%matplotlib inline

x = np.arange(-5, 5, 0.3)[:32].reshape((32, 1))
y = -5 * x + 0.1 * np.random.normal(loc=0.0, scale=20.0, size=x.shape)
plt.scatter(x, y, label='Samples')
plt.plot(x, w * x + b, c='r', label='Trained model')
plt.plot(x, -5 * x + 0.1, c='b', label='True function')
plt.legend()
```

输出：

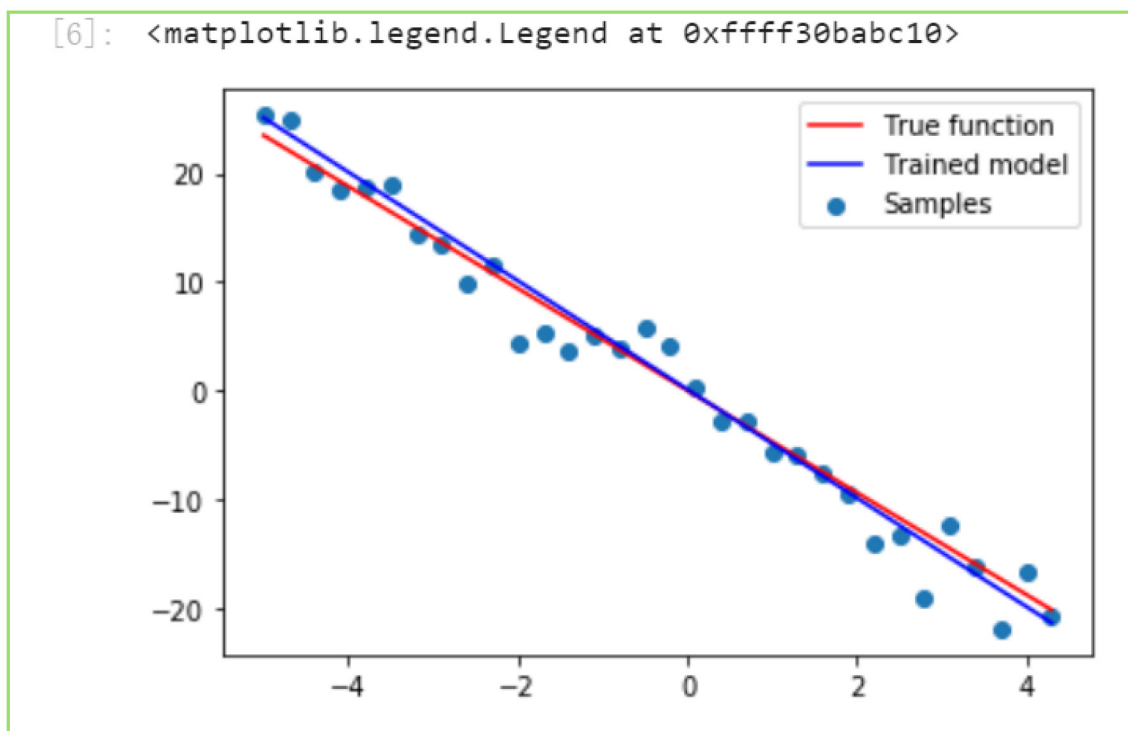


图2-1 模拟数据展示

2.4 实验小结

本实验主要介绍使用 MindSpore 在模拟数据上进行线性回归实验，分析自变量和因变量之间的线性关系，即求得一个线性函数。